

数据结构与算法作业

第一章

题目一：在软件开发中，具备相同功能的软件，不同的实现往往具有性能上的不同。请从数据结构的角度分析该现象。

在开发功能相似的软件时，我们常常会发现，不同程序员写出来的程序在运行速度、内存消耗上有着明显的区别。这背后一个非常关键的原因，就在于对数据结构的选择不同。就像去同一个目的地，选择步行、骑车还是开车，效率和体验是完全不同的。下面我将从几个方面来分析数据结构是如何影响软件性能的。

一、时间效率的差异

数据结构最直接的影响就体现在程序运行的速度上。对于同一个操作，不同的数据结构带来的时间成本可能天差地别。

比如，我们要在一个系统中根据姓名查找用户信息。如果程序采用数组或链表来存储所有用户，那么在最坏的情况下，就需要从头到尾遍历整个列表，才能找到目标用户。假设用户数量有十万个，那么平均就需要进行五万次比较，这个速度显然是不能令人满意的。

但如果我们换一种方式，使用哈希表来存储这些用户信息，情况就大不相同了。哈希表可以通过一个特定的函数，直接将姓名“计算”成一个地址，然后近乎直接地定位到该用户的数据。这种操作在理想情况下只需要常数时间，与用户总数无关。对于十万用户，查找一次的时间和查找一百个用户的时间几乎是差不多的，效率的提升是巨大的。

二、空间利用的权衡

除了运行速度，内存的使用情况也是衡量性能的重要指标。有时候，为了追求更快的速度，我们不得不在空间上做出一些牺牲，这被称为“以空间换时间”。

一个典型的例子就是链表。链表中的每个节点，除了要存储实际的数据外，还必须附带一个或两个指针，用来指向下一个或上一个节点。这些指针本身不存储业务数据，却要占用额外的内存空间。相比之下，数组的所有元素都紧密地排列在一起，没有这种额外的开销，内存利用率更高。

再比如，在处理一个绝大部分元素都是 0 的矩阵（稀疏矩阵）时，如果用一个标准的二维数组来存储，就会浪费大量内存去记录那些无意义的 0。而如果采用“三元组顺序表”或者“十字链表”这样的专用结构，只存储非零元素的位置和值，就可以节省下大量的内存空间。

三、数据操作与场景的匹配

不同的数据结构天生就适合处理不同类型的操作。如果选错了结构，就好像用勺子去吃面条，虽然也能吃，但效率非常低下。

栈这种结构特别适合处理需要“回溯”或“撤销”的场景，比如浏览器页面的前进后退、程序执行时的函数调用；而队列则完美契合了“先来后到”的排队场景，比如打印任务队列、消息队列等。

又比如，如果需要频繁地在数据集中进行插入和删除操作，链表通常比数组表现更好，因为链表只需要修改指针，而数组可能需要进行大量元素的移动。但如果需要频繁地按位置随机访问元素，那么支持“下标访问”的数组又比链表高效得多。

四、缓存友好性的影响

在现代计算机体系中，数据访问的速度并不均等。CPU 访问缓存的速度远快于访问内存。因此，能够更好地利用 CPU 缓存的数据结构，其实际性能往往会更好。

数组的元素在内存中是连续存储的，当我们访问第一个元素时，计算机通常会预加载其后面的一片数据到高速缓存中。这样，当我们接下来访问后续元素时，就可以直接从高速缓存中读取，速度极快。这种特性被称为“空间局部性”好。

而链表则相反，它的节点在内存中往往是分散的、不连续的。访问一个节点后，并不能保证下一个节点已经被加载到缓存中，很可能需要再次去内存中寻找。这种频繁的缓存不命中，会显著拖慢程序的真实运行速度，即便它的理论时间复杂度可能与数组相同。

五、总结

总而言之，数据结构的选择绝非无关紧要，它直接决定了软件在时间和空间上的基本效率。一个精心选择的数据结构，能够让代码在处理大规模数据时依然保持敏捷；而一个不当的选择，则可能让程序举步维艰。因此，在软件开发中，我们必须根据具体的应用场景、数据规模和操作特点，来慎重地选择最合适的数据结构，这正是在实现相同功能时，性能表现千差万别的根本原因之一。

第二章

题目一：

(1) 创建新链表 B 的头结点→用两个指针 p 和 q 分别指向链表 A 和 B 的当前尾结点→从头结点开始遍历原链表 A，使用计数器记录结点序号→序号为奇数的结点保留在 A 链中，序号为偶数的结点移动到 B 链中→保持两个链表中结点的相对顺序不变→最后设置两个链表的尾指针为 NULL

(2)

```
#include <iostream>
using namespace std; // 定义单链表结点结构
typedef struct LNode {
    int data;          // 数据域
    struct LNode* next; // 指针域
} LNode, * LinkList; // 尾插法建立单链表
void CreateList_R(LinkList& L, int arr[], int n) {
    L = new LNode;      // 创建头结点
    L->next = NULL;
    LNode* r = L;       // 尾指针
    for (int i = 0; i < n; i++) {
        LNode* p = new LNode;
        p->data = arr[i];
        p->next = NULL;
        r->next = p;
        r = p;          // 更新尾指针
    }
}
```

```

} // 分解链表算法
void SplitList(LinkList& A, LinkList& B) {
    B = new LNode;    // 创建 B 链表的头结点
    B->next = NULL;

    LNode* p = A->next; // p 指向 A 链第一个数据结点
    LNode* ra = A;      // ra 指向 A 链的尾结点（初始为头结点）
    LNode* rb = B;      // rb 指向 B 链的尾结点
    int count = 1;      // 结点序号计数器
    while (p != NULL) {
        if (count % 2 == 1) { // 奇数序号结点
            ra->next = p;    // 连接到 A 链尾部
            ra = p;          // 更新 A 链尾指针
        }
        else {               // 偶数序号结点
            rb->next = p;    // 连接到 B 链尾部
            rb = p;          // 更新 B 链尾指针
        }
        p = p->next;        // 移动到下一个结点
        count++;
    } // 设置两个链表的尾结点 next 为 NULL
    ra->next = NULL;
    rb->next = NULL;
} // 打印链表
void PrintList(LinkList L) {
    LNode* p = L->next;
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    LinkList A, B;
    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int n = sizeof(arr) / sizeof(arr[0]); // 创建原始链表 A
    CreateList_R(A, arr, n);
    cout << "原始链表 A: ";
    PrintList(A); // 分解链表
    SplitList(A, B);
    cout << "分解后链表 A(奇数序号): ";
    PrintList(A);
    cout << "分解后链表 B(偶数序号): ";
    PrintList(B);
}

```

```

    return 0;
}

```

原始链表A: 1 2 3 4 5 6 7 8 9 10
 分解后链表A(奇数序号): 1 3 5 7 9
 分解后链表B(偶数序号): 2 4 6 8 10

(3) 时间复杂度: $O(n)$

总时间 = $O(1) + n \times O(1) = O(n)$

空间复杂度: $O(1)$

总空间 = 常数空间 = $O(1)$

(4) 任务调度: 在操作系统中, 可以将就绪队列中的进程按优先级分组, 奇数序号的高优先级进程和偶数序号的低优先级进程分别调度; 数据备份: 在数据库系统中, 可以将事务日志按奇偶分开存储, 实现数据的冗余备份, 提高系统可靠性。

题目二:

(1) 将数组 R 看作由两部分组成: 前 p 个元素和后 n-p 个元素 → 先反转前 p 个元素 → 再反转后 n-p 个元素 → 最后反转整个数组 → 经过三次反转后, 就实现了循环左移 p 位的效果

(2)

```
#include <iostream>
```

```
using namespace std; // 反转数组的指定区间
```

```
void Reverse(int R[], int left, int right) {
```

```
    while (left < right) {
        // 交换左右两端的元素
        int temp = R[left];
        R[left] = R[right];
        R[right] = temp;
        left++;
        right--;
    }
```

```
} // 循环左移 p 位的主函数
```

```
void LeftShift(int R[], int n, int p) {
```

```
    if (p <= 0 || p >= n) {
        return; // 不需要移动或移动位数无效
    } // 三次反转法
```

```
    Reverse(R, 0, p - 1); // 反转前 p 个元素
    Reverse(R, p, n - 1); // 反转后 n-p 个元素
    Reverse(R, 0, n - 1); // 反转整个数组
```

```
} // 打印数组
```

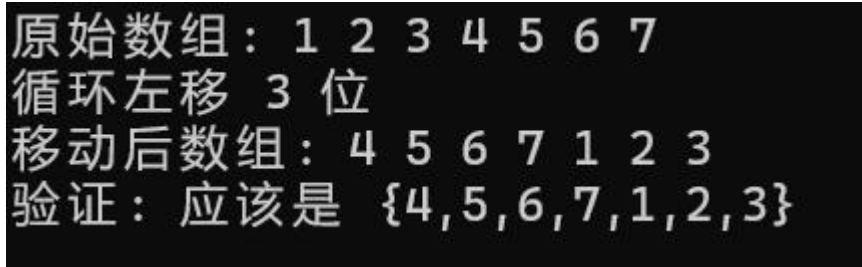
```
void PrintArray(int R[], int n) {
```

```
    for (int i = 0; i < n; i++) {
        cout << R[i] << " ";
    }
```

```

        cout << endl;
    }
    int main() {
        const int n = 7;
        int R[n] = { 1, 2, 3, 4, 5, 6, 7 };
        int p = 3;
        cout << "原始数组: ";
        PrintArray(R, n);
        cout << "循环左移 " << p << " 位" << endl;
        LeftShift(R, n, p);
        cout << "移动后数组: ";
        PrintArray(R, n);
        // 验证结果
        cout << "验证: 应该是 {4,5,6,7,1,2,3}" << endl;
        return 0;
    }

```



```

原始数组: 1 2 3 4 5 6 7
循环左移 3 位
移动后数组: 4 5 6 7 1 2 3
验证: 应该是 {4,5,6,7,1,2,3}

```

(3) 时间复杂度: $O(n)$

总交换次数 = $p/2 + (n-p)/2 + n/2 = n$ 次交换

每个交换操作是 $O(1)$, 所以总时间复杂度是 $O(n)$

空间复杂度: $O(1)$

没有使用额外的数组或其他数据结构, 所有操作都在原数组上进行

(4) 在实时操作系统中, 采用轮转调度算法时, 任务队列的调度本质上就是循环左移操作; 在网络数据包处理或实时数据流分析中, 经常需要维护一个固定大小的数据窗口。当新数据到达时, 需要移除最旧的数据并加入新数据, 这本质上就是循环左移操作。

第三章

题目一:

(1) 缓冲区的逻辑结构

从数据结构视角分析, 打印机缓冲区采用的是队列结构

原因: 主机写入数据的顺序与打印机读取数据的顺序一致; 先进入缓冲区的文件先被打印, 后进入的后被打印; 队列天然具有缓冲功能, 可以平衡速度不匹配的问题

(2) 打印顺序: 文件 A → 文件 B → 文件 C → 文件 D

题目二:

(1) B → D → C → E → A

B → D → C → A → E

B → D → E → C → A

(2) 现实生活中的栈现象：食堂餐盘堆叠和手枪弹匣装弹
软件开发中的栈应用：浏览器后退功能和文本编辑器的撤销操作

题目三：

(1) 递归算法：

汉诺塔问题可以通过递归分解为子问题。假设有 n 个盘子，从 A 柱移动到 C 柱，使用 B 柱作为辅助：

如果 $n=1$ ，直接将盘子从 A 移到 C

如果 $n>1$ ，则将 $n-1$ 个盘子从 A 移到 B（使用 C 作为辅助）

将第 n 个盘子从 A 移到 C

将 $n-1$ 个盘子从 B 移到 C（使用 A 作为辅助）

非递归算法：

初始化栈，将初始任务 (n, A, C, B) 入栈→

循环直到栈为空：弹出栈顶任务：如果任务中盘子数为 1，直接移动，否则，将子任务按相反顺序入栈（确保执行顺序正确）→将第三步任务入栈：移动 $n-1$ 从辅助柱到目标柱使用源柱→将第二步任务入栈：移动 1 个盘子从源柱到目标柱→将第一步任务入栈：移动 $n-1$ 从源柱到辅助柱使用目标柱

(2) #include <iostream>

#include <stack>

using namespace std;

// 递归实现汉诺塔

void hanoiRecursive(int n, char from, char to, char aux) {

if (n == 1) {

cout << "移动盘子 " << n << " 从 " << from << " 到 " << to << endl;

return;

}

hanoiRecursive(n - 1, from, aux, to);

cout << "移动盘子 " << n << " 从 " << from << " 到 " << to << endl;

hanoiRecursive(n - 1, aux, to, from);

}

// 非递归实现汉诺塔

struct Task {

int n; // 盘子数量

char from; // 源柱

char to; // 目标柱

char aux; // 辅助柱

};

void hanoiIterative(int n, char from, char to, char aux) {

stack<Task> st;

st.push({ n, from, to, aux }); // 初始任务

while (!st.empty()) {

Task task = st.top();

st.pop();

```

        if (task.n == 1) {
            cout << "移动盘子 " << task.n << " 从 " << task.from << " 到 " <<
task.to << endl;
        }
        else {
            // 将三个子任务按相反顺序入栈（因为栈是 LIFO）
            // 第一步：移动 n-1 从 aux 到 to 使用 from
            st.push({ task.n - 1, task.aux, task.to, task.from });
            // 第二步：移动 1 个盘子从 from 到 to
            st.push({ 1, task.from, task.to, task.aux });
            // 第三步：移动 n-1 从 from 到 aux 使用 to
            st.push({ task.n - 1, task.from, task.aux, task.to });
        }
    }
}

int main() {
    int n = 3; // 盘子数量
    cout << "递归解法： " << endl;
    hanoiRecursive(n, 'A', 'C', 'B');
    cout << endl;
    cout << "非递归解法： " << endl;
    hanoiIterative(n, 'A', 'C', 'B');
    return 0;
}

```

```

递归解法：
移动盘子 1 从 A 到 C
移动盘子 2 从 A 到 B
移动盘子 1 从 C 到 B
移动盘子 3 从 A 到 C
移动盘子 1 从 B 到 A
移动盘子 2 从 B 到 C
移动盘子 1 从 A 到 C

非递归解法：
移动盘子 1 从 A 到 C
移动盘子 1 从 A 到 B
移动盘子 1 从 C 到 B
移动盘子 1 从 A 到 C
移动盘子 1 从 B 到 A
移动盘子 1 从 B 到 C
移动盘子 1 从 A 到 C

```

(3) 递归算法:

时间复杂度: $O(2^n)$

移动次数满足递推关系 $T(n) = 2T(n-1) + 1$, 解为 $T(n) = 2^n - 1$, 因此时间复杂度为 $O(2^n)$

空间复杂度: $O(n)$

递归调用栈的深度为 n , 每层递归需要常数空间存储参数, 因此空间复杂度为 $O(n)$

非递归算法:

时间复杂度: $O(2^n)$

空间复杂度: $O(n)$

栈中最多同时存储约 $3n$ 个任务, 因此空间复杂度为 $O(n)$

第四章

题目一:

(1) 基本原理: 压缩存储技术是通过减少数据冗余来节省存储空间的方法。对于颜色单一的图片, 其对应的矩阵中存在大量相同的元素, 压缩存储就是利用这种数据的规律性和重复性, 只存储必要的信息。

压缩存储的目的: 节省存储空间, 减少数据传输时间, 提高数据处理效率, 降低存储成本。

(2) 对称矩阵压缩存储:

```
#include <iostream>
using namespace std;
class SymmetricMatrix {
private:
    int* data;
    int n;
public:
    SymmetricMatrix(int matrix[][3], int size) {
        n = size;
        int length = n * (n + 1) / 2;
        data = new int[length];
        int index = 0;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j <= i; j++) {
                data[index++] = matrix[i][j];
            }
        }
    }
    int get(int i, int j) {
        if(i < 0 || i >= n || j < 0 || j >= n) {
            cout << "索引越界" << endl;
            return -1;
        }
    }
}
```



```

        if(i >= j) {
            return data[i * (i + 1) / 2 + j];
        } else {
            return data[j * (j + 1) / 2 + i];
        }
    }

// 打印矩阵
void printMatrix() {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            cout << get(i, j) << "\t";
        }
        cout << endl;
    }
}

void printCompressed() {
    cout << "压缩存储数组: ";
    int length = n * (n + 1) / 2;
    for(int i = 0; i < length; i++) {
        cout << data[i] << " ";
    }
    cout << endl;
}

// 计算压缩率
void compressionRate() {
    int original_size = n * n * sizeof(int);
    int compressed_size = (n * (n + 1) / 2) * sizeof(int);
    double rate = (1.0 - (double)compressed_size / original_size) * 100;
    cout << "原始大小: " << original_size << " 字节" << endl;
    cout << "压缩后大小: " << compressed_size << " 字节" << endl;
    cout << "压缩率: " << rate << "%" << endl;
}

~SymmetricMatrix() {
    delete[] data;
}

};

int main() {
    int matrix[3][3] = {
        {0, 12, 13},
        {12, 0, 23},

```

```

        {13, 23, 0}
    };

    cout << "原始矩阵:" << endl;
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }
    SymmetricMatrix compressed(matrix, 3);
    cout << "\n 从压缩数据恢复的矩阵:" << endl;
    compressed.printMatrix();
    compressed.printCompressed();
    compressed.compressionRate();
    return 0;
}

```

```

原始矩阵 :
0      12      13
12      0      23
13      23      0

从压缩数据恢复的矩阵 :
0      12      13
12      0      23
13      23      0
压缩存储数组 : 0 12 0 13 23 0
原始大小 : 36 字节
压缩后大小 : 24 字节
压缩率 : 33.3333%

```

题目二:

(1) 三元组顺序表存储:

```

#include <iostream>
using namespace std;
const int MAXSIZE = 100;
struct Triple {
    int row, col, value;
};
class SparseMatrix {
private:
    Triple data[MAXSIZE];
    int rows, cols, terms;

```

```

public:
    SparseMatrix(int mat[6][6], int r, int c) {
        rows = r;
        cols = c;
        terms = 0;
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                if(mat[i][j] != 0) {
                    data[terms].row = i;
                    data[terms].col = j;
                    data[terms].value = mat[i][j];
                    terms++;
                }
            }
        }
    }

    void printCompressed() {
        cout << "稀疏矩阵压缩存储（三元组表示）： " << endl;
        cout << "行\t 列\t 值" << endl;
        for(int i = 0; i < terms; i++) {
            cout << data[i].row << "\t" << data[i].col << "\t" << data[i].value <<
endl;
        }
        cout << "矩阵大小： " << rows << "x" << cols << endl;
        cout << "非零元素个数： " << terms << endl;
    }

    void printMatrix() {
        cout << "原始矩阵： " << endl;
        int index = 0;
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                if(index < terms && data[index].row == i && data[index].col == j)
{
                    cout << data[index].value << "\t";
                    index++;
                } else {
                    cout << "0\t";
                }
            }
            cout << endl;
        }
    }
};

```

```

(2) #include <iostream>
using namespace std;
const int MAX_TERMS = 20;
const int MAX_COLS = 6; // 定义最大列数
struct Triple {
    int row, col, value;
};
class SparseMatrix {
public:
    Triple data[MAX_TERMS]; // 三元组数组
    int rows, cols, terms; // 行数、列数、非零元个数
    // 构造函数，初始化成员变量
    SparseMatrix() : rows(0), cols(0), terms(0) {
        // 初始化三元组数组
        for (int i = 0; i < MAX_TERMS; i++) {
            data[i].row = 0;
            data[i].col = 0;
            data[i].value = 0;
        }
    }
    // 创建稀疏矩阵
    void create() {
        rows = 6; cols = 6; terms = 7;

        // 初始化三元组
        data[0] = { 0,0,-5 }; data[1] = { 0,1,-2 };
        data[2] = { 1,3,-6 }; data[3] = { 3,1,-3 };
        data[4] = { 4,0,-7 }; data[5] = { 4,3,-4 };
        data[6] = { 5,2,-1 };
    }
    // 快速转置算法
    SparseMatrix fastTranspose() {
        SparseMatrix result;
        result.rows = cols; // 转置后行数=原列数
        result.cols = rows; // 转置后列数=原行数
        result.terms = terms; // 非零元个数不变
        if (terms == 0) return result;
        // 步骤 1: 统计每列的非零元素个数
        int colCount[MAX_COLS] = { 0 }; // 使用固定大小数组
        for (int i = 0; i < terms; i++) {
            colCount[data[i].col]++;
        }
        // 步骤 2: 计算转置后每行的起始位置
        int startPos[MAX_COLS] = { 0 }; // 使用固定大小数组
    }
};

```

```

        for (int i = 1; i < cols; i++) {
            startPos[i] = startPos[i - 1] + colCount[i - 1];
        }
        // 步骤 3: 进行转置
        for (int i = 0; i < terms; i++) {
            int j = startPos[data[i].col]; // 找到正确位置
            result.data[j].row = data[i].col; // 行变列
            result.data[j].col = data[i].row; // 列变行
            result.data[j].value = data[i].value; // 值不变
            startPos[data[i].col]++; // 该行位置后移
        }
        return result;
    }
    // 打印矩阵
    void print() {
        cout << "矩阵: " << endl;
        int index = 0;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (index < terms && data[index].row == i && data[index].col == j)
                {
                    cout << data[index].value << "\t";
                    index++;
                }
                else {
                    cout << "0\t";
                }
            }
            cout << endl;
        }
    }
    // 打印三元组
    void printTriples() {
        cout << "三元组表: " << endl;
        cout << "行\t列\t值" << endl;
        for (int i = 0; i < terms; i++) {
            cout << data[i].row << "\t" << data[i].col << "\t" << data[i].value <<
endl;
        }
    }
};

int main() {
    SparseMatrix sm;
    sm.create();

```

```

    cout << "=== 转置前 ===" << endl;
    sm.print();
    sm.printTriples();
    cout << "\n=== 转置后 ===" << endl;
    SparseMatrix transposed = sm.fastTranspose();
    transposed.print();
    transposed.printTriples();
    return 0;
}

```

行	列	值
0	0	-5
0	1	-2
1	3	-6
3	1	-3
4	0	-7
4	3	-4
5	2	-1

=== 转置后 ===

矩阵：

-5	0	0	0	-7	0
-2	0	0	-3	0	0
0	0	0	0	0	-1
0	-6	0	0	-4	0
0	0	0	0	0	0
0	0	0	0	0	0

三元组表：

行	列	值
0	0	-5
0	4	-7
1	0	-2
1	3	-3
2	5	-1
3	1	-6
3	4	-4

第五章

题目一：

（1）设计方案：

采用最大堆数据结构来存储邮件。最大堆是一种完全二叉树，其中每个节点的值都大于或等于其子节点的值，这样堆顶元素始终是当前重要程度最高的邮件。具体步骤如下：将每个邮件的重要程度作为元素，依次插入最大堆中。插入操作需要调整堆结构以维持最大堆性质。每次从堆顶取出重要程度最高的邮件进行处理，取出后调整堆结构，确保堆顶仍是剩余邮件中重要程度最高的。

时间复杂度分析：

使用依次插入的方法，构建堆的总时间复杂度为 $O(n \log n)$ 。

（2） $51 \rightarrow 31 \rightarrow 18 \rightarrow 17 \rightarrow 12 \rightarrow 9$

所以为 12 和 9

题目二：

n_0 为叶节点数， n_1 为非叶节点数，设 n 为总结点数， $n=n_0+n_1$

总边数： $k \times n_1 = n - 1$

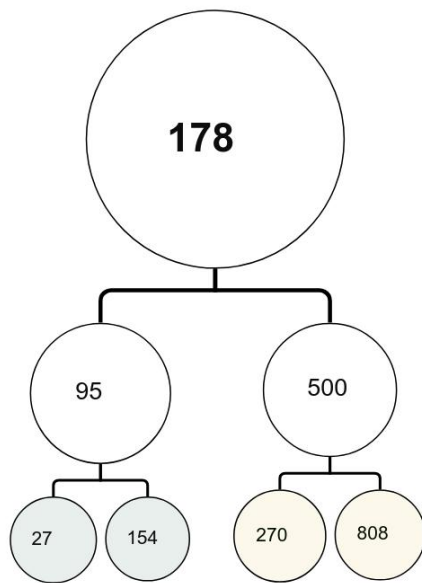
代入 $n=n_0+n_1$ ： $k \times n_1 = (n_0+n_1) - 1$

整理得： $n_0 = (k-1) n_1 + 1$

第六章

题目一：

(1)



(2) 178,500,808

(3) 查找成功： $(1+2 \times 2+3 \times 4) \div 7=2.4286$

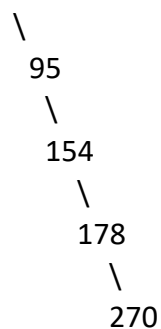
查找不成功：3 次

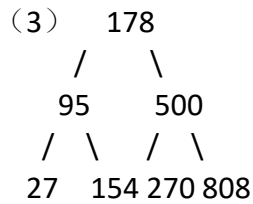
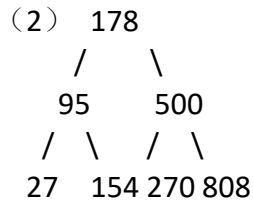
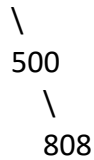
(4) 有序顺序表的静态查找，例如数据库索引

第七章

题目一：

(1) 27





(4) 检索故障代码 500 时的区别

二叉排序树：需要从根节点 27 开始，依次与 27、95、154、178、270、500 比较，共 6 次比较。

二叉平衡树：从根节点 178 开始，依次与 178、500 比较，共 2 次比较。

第八章

题目一：

(1) 取 $\text{mod}=7$ ，冲突处理：线性探测法

索引	0	1	2	3	4	5	6
学号	70	35		80	25	60	
姓名	A	D		C	B	E	
成绩	优	中		中	良	差	

(2) 2 次

(3) $(1 \times 3 + 2 \times 2) \div 5 = 1.4$

(4) 哈希函数的均匀性：好的哈希函数应使关键字均匀分布在哈希表中，减少冲突。

冲突处理方法：不同方法对查找性能有直接影响。线性探测易产生聚集，而链地址法可减少聚集。

装填因子 α ： $\alpha = \text{表中记录数} / \text{哈希表长度}$ ， α 越大，冲突概率越高，查找性能下降。通常需控制 α 在合理范围。

哈希表大小：表大小过小易冲突，过大浪费空间。选择质数作为表长有助于均匀分布。

关键字的分布：若关键字本身分布不均，即使好的哈希函数也可能产生较多冲突。

第九章

题目一：

(1) B-F-A-E-D-C-G

总费用：14

第十章

题目一：

快速排序过程：

初始序列：[1, 2, 3, 4, 9, 8, 7]

[1] [2, 3, 4, 9, 8, 7]

[1, 2] [3, 4, 9, 8, 7]

[1, 2, 3] [4, 9, 8, 7]

[1, 2, 3, 4] [9, 8, 7]

[1, 2, 3, 4, 7, 8, 9]

[1, 2, 3, 4, 7, 8, 9]

最终有序序列：[1, 2, 3, 4, 7, 8, 9]

时间复杂度：

平均情况： $O(n \log n)$

最坏情况： $O(n^2)$ 。本题出现最坏情况，空间复杂度为 $O(n^2)$ 。

空间复杂度：

平均情况：递归深度为 $O(\log n)$ ，空间复杂度为 $O(\log n)$ 。

最坏情况：递归深度为 $O(n)$ ，空间复杂度为 $O(n)$ 。本题出现最坏情况，空间复杂度为 $O(n)$ 。